

Procesamiento del Lenguaje Natural I

TEMA 5. TRANSFORMERS (III)

Soto Montalvo Herranz

Objetivos

- Aprender la importancia de la tokenización en transformers.

Tokenización en transformers

- La tokenización es crucial en el caso de los modelos transformers.
 - Se ocupa de representar los datos en un formato que entienda el modelo.
 - Cada token se representa con un número (índice del token en el vocabulario del transformer)
 - Cada token se transforma a un vector de números reales (entrada al modelo).
- Si se va a utilizar un modelo preentrenado **hay que utilizar el tokenizar asociado al modelo.**
 - Hay que dividir el texto siguiendo las mismas reglas que se utilizaron al preprocesar los textos empleados en el preentrenamiento.
 - Se tienen que utilizar los mismos índices para representar los tokens como números.

Tokenización en transformers

- Carga del tokenizador
 - Se utilizará la clase AutoTokenizer (carga el tokenizador de cualquier transformer)
 - En el ejemplo se está cargando el modelo multilingüe de BERT

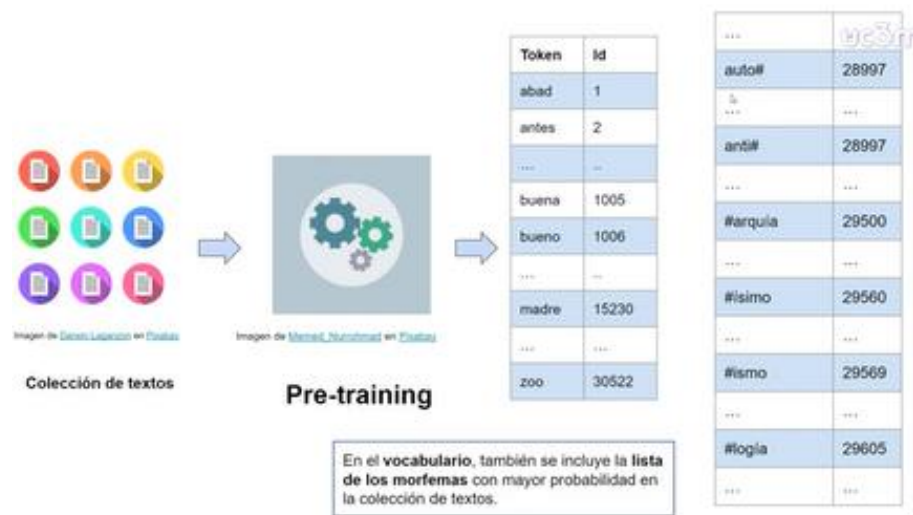
```
from transformers import AutoTokenizer  
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
```

- Salida del tokenizador de BERT

```
example = tokenizer("Estamos haciendo una prueba de tokenización con un modelo preentrenado")  
print(example)  
  
{'input_ids': [101, 9765, 22591, 2015, 5292, 23402, 15482, 14477, 10975, 5657, 3676, 2139, 19204, 21335, 10446, 9530, 4895, 2944, 2080, 365, 102], 'token_type_ids': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'attention_mask': [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]}
```

Tokenización en transformers

- Wordpiece tokenization



Imágenes: [1]



Ventaja del wordpiece: se podrá representar cualquier texto.

Tokenización en transformers

- Ejemplo con versión de BERT multilingüe

```
from transformers import BertTokenizer
bert_tokenizer = BertTokenizer.from_pretrained("bert-base-multilingual-uncased")

sentence = 'Esto es supercalifristicoespialidoso'
encoding = bert_tokenizer(sentence)
print(encoding)

{'input_ids': [101, 15195, 10153, 12278, 17513, 14808, 76314, 12965, 19781, 80343, 11513, 102], 'token_type_id': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], 'attention_mask': [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]}
```

```
tokens = bert_tokenizer.convert_ids_to_tokens(encoding.input_ids)
print(tokens)

['[CLS]', 'esto', 'es', 'super', '##cal', '##if', '##risti', '##coes', '##pia', '##lido', '##so', '[SEP]']
```

- WordPiece se utiliza por modelos como BERT, DistillBert, . Existen otros algoritmos similares, por ejemplo, GPT utiliza una variación de WordPiece

Tokenización en transformers

- Campo input_ids

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("bert-base-multilingual-uncased")
```

```
encoded = tokenizer("Ejemplo sencillo para ver la tokenización con el modelo BERT")
#encoded
```

```
print("input_ids: ", encoded.input_ids)
print("token_type_ids: ", encoded.token_type_ids)
print("attention_mask: ", encoded.attention_mask)
```

```
input_ids: [101, 20244, 32593, 10239, 15405, 10106, 16925, 13649, 21585, 10173, 10117, 19102, 40668, 102]
token_type_ids: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
attention_mask: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

Tokenización en transformers

- Campo input_ids

```
encoded = tokenizer('Ven')
print("input_ids: ", encoded.input_ids)
print("token_type_ids: ", encoded.token_type_ids)
print("attention_mask: ", encoded.attention_mask)
```

```
input_ids: [101, 22712, 102]
token_type_ids: [0, 0, 0]
attention_mask: [1, 1, 1]
```

101 es el id de 'CLS',
102 es el id de 'SEP'

```
tokens = tokenizer.convert_ids_to_tokens(encoded.input_ids)
print(tokens)
```

```
['[CLS]', 'ven', '[SEP]']
```


Tokenización en transformers

- Campo token_type_ids
 - Bert necesita este campo porque ha sido entrenado utilizando dos estrategias
 - Enmascaramiento de tokens
 - Clasificación: Next Sentence Prediction (NSP): tiene que identificar si dos oraciones aparecen juntas en un texto o no.



Imagen: [1]

Tokenización en transformers

- Campo token_type_ids

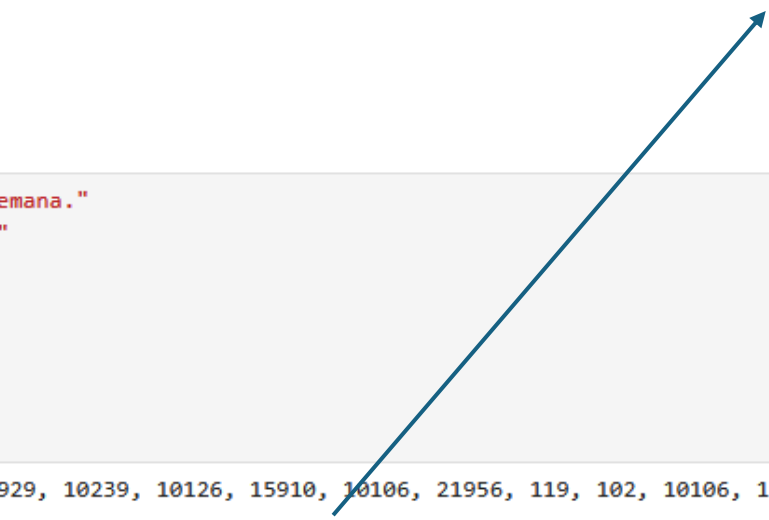
Tokens de la segunda oración

```
sentence1 = "Hoy es jueves, queda poco para que termine la semana."  
sentence2 = "La asignatura que más me gusta del grado es PLN"
```

```
encoded = tokenizer(sentence1, sentence2)
```

```
print("input_ids: ", encoded.input_ids)  
print("token_type_ids: ", encoded.token_type_ids)  
print("attention_mask: ", encoded.attention_mask)
```

```
input_ids: [101, 21150, 10153, 12955, 32324, 117, 26173, 13929, 10239, 10126, 15910, 10106, 21956, 119, 102, 10106, 12179, 23464, 14135, 10126, 10321, 105  
25, 49475, 10211, 10134, 18663, 10153, 16360, 10115, 102]  
token_type_ids: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]  
attention_mask: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```



```
tokens = tokenizer.convert_ids_to_tokens(encoded.input_ids)  
print(tokens)
```

```
['[CLS]', 'hoy', 'es', 'ju', '##eves', ',', 'queda', 'poco', 'para', 'que', 'termine', 'la', 'semana', '.', '[SEP]', 'la', 'asi', '##gna', '##tura', 'que',  
'mas', 'me', 'gus', '##ta', 'del', 'grado', 'es', 'pl', '##n', '[SEP]']
```

Bibliografía

- [1] Segura Bedmar, I. Transformers. Procesamiento del Lenguaje Natural con Aprendizaje Profundo. Máster en Ciencia y Tecnología Informática. 2023. ([Enlace](#))